

Writeup PWN - CyLab: Heap0

Nguyễn Lê Anh Tuấn

1. Thông tin ban đầu (Reconnaissance)

Đầu tiên, chúng ta sẽ kiểm tra các thông tin cơ bản của file thực thi bằng lệnh `file`:

```
$ file chall
chall: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[
sha1]=2015ade3c2b89f5069cb8c54dd750d1b9849062d, for GNU/Linux 3.2.0,
with debug_info, not stripped
```

Output của lệnh `file`

Tiếp theo, ta sử dụng công cụ `checksec` để kiểm tra toàn diện các cơ chế bảo vệ. Thông tin chi tiết được cung cấp trong bảng dưới đây:

File	NX	PIE	Canary	Relro	RPATH	RUNPATH	Symbols	FORTIFY	Fortified	Fortifiable	Fortify Score
chall	Yes	Yes	No	Partial	No	No	Yes	No	No	No	0

Phân tích các cơ chế bảo vệ cốt lõi:

Dù công cụ trả về khá nhiều thông số, nhưng đối với bài Heap0 này, chúng ta chỉ cần quan tâm đến 4 cơ chế ảnh hưởng trực tiếp đến hướng tấn công:

- **Relro (Partial):** Bảng GOT (Global Offset Table) có quyền ghi. Điều này mở ra hướng tấn công GOT Overwrite nếu ta có lỗ hổng ghi tùy ý (Arbitrary Write).
- **Canary (No):** Không có Canary bảo vệ trên Stack. Việc thực hiện Buffer Overflow sẽ không bị chương trình phát hiện và báo lỗi `stack smashing detected`.
- **NX (Yes):** Stack và Heap không có quyền thực thi. Ta không thể đẩy trực tiếp shellcode vào vùng `input_data` để chạy, mà phải tấn công bằng cách ghi đè biến `safe_var` có sẵn.
- **PIE (Yes):** Địa chỉ các section (`.text`, `.bss`, `.data`) bị xáo trộn ngẫu nhiên mỗi lần chạy. Ta không thể hardcode các địa chỉ cố định vào script exploit.

2. Phân tích mã nguồn

```
void check_win() {
    if (strcmp(safe_var, "bico") != 0) {
        printf("\nYOU WIN\n");

        // Print flag
        char buf[FLAGSIZE_MAX];
        FILE *fd = fopen("flag.txt", "r");
        fgets(buf, FLAGSIZE_MAX, fd);
        printf("%s\n", buf);
        fflush(stdout);

        exit(0);
    } else {
        printf("Looks like everything is still secure!\n");
        printf("\nNo flage for you :(\n");
        fflush(stdout);
    }
}
```

Hình 1: Mã nguồn hàm check_win() và init() trong Heap0

Khi đọc mã nguồn, ta xác định được hai điểm then chốt:

Điều kiện để in flag: Hàm check_win() sử dụng điều kiện:

```
1 if (strcmp(safe_var, "bico") != 0) {
2     // In ra flag
3     puts("YOU WIN");
4     system("cat flag.txt");
5 }
```

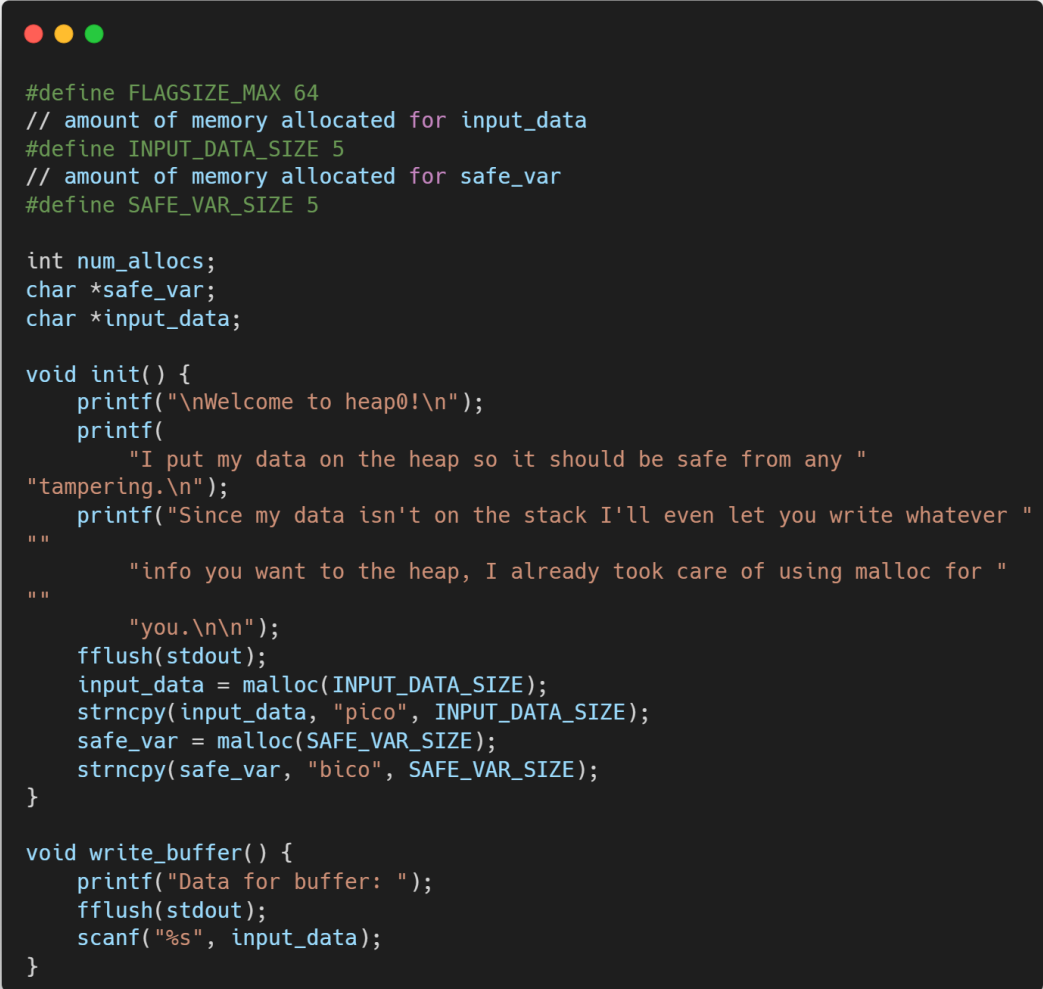
Nghĩa là flag chỉ được in khi `safe_var` khác chuỗi "bico". Ngược lại, nếu `safe_var == "bico"` thì chương trình thông báo hệ thống vẫn an toàn. Mục tiêu của người khai thác là thay đổi giá trị `safe_var` thành bất kỳ chuỗi nào khác "bico".

Cách cấp phát bộ nhớ: Trong hàm init(), cả hai biến toàn cục `input_data` và `safe_var` đều được cấp phát trên heap:

```
1 #define INPUT_DATA_SIZE 5
2 #define SAFE_VAR_SIZE 5
3
4 input_data = malloc(INPUT_DATA_SIZE); // 5 byte
5 safe_var = malloc(SAFE_VAR_SIZE); // 5 byte
6
7 strcpy(input_data, "pico");
8 strcpy(safe_var, "bico");
```

Mỗi vùng chỉ được cấp phát 5 byte – đủ để chứa 4 ký tự và ký tự null kết thúc chuỗi.

3. Nguyên nhân lỗi hỏng



```
#define FLAGSIZE_MAX 64
// amount of memory allocated for input_data
#define INPUT_DATA_SIZE 5
// amount of memory allocated for safe_var
#define SAFE_VAR_SIZE 5

int num_allocs;
char *safe_var;
char *input_data;

void init() {
    printf("\nWelcome to heap0!\n");
    printf(
        "I put my data on the heap so it should be safe from any "
        "tampering.\n");
    printf("Since my data isn't on the stack I'll even let you write whatever "
        ""
        "info you want to the heap, I already took care of using malloc for "
        ""
        "you.\n\n");
    fflush(stdout);
    input_data = malloc(INPUT_DATA_SIZE);
    strncpy(input_data, "pico", INPUT_DATA_SIZE);
    safe_var = malloc(SAFE_VAR_SIZE);
    strncpy(safe_var, "bico", SAFE_VAR_SIZE);
}

void write_buffer() {
    printf("Data for buffer: ");
    fflush(stdout);
    scanf("%s", input_data);
}
```

Hình 2: Hàm `write_buffer()` chứa lỗi hỏng heap overflow

Trong hàm `write_buffer()`, chương trình đọc input của người dùng bằng:

```
1 scanf("%s", input_data);
```

Lệnh `scanf("%s")` đọc đến khi gặp ký tự trắng (space, newline) và không có giới hạn độ dài. Nếu người dùng nhập chuỗi dài hơn 5 byte, dữ liệu sẽ ghi vượt qua ranh giới của `input_data` và tràn sang các chunk lân cận trên Heap.

4. Layout bộ nhớ heap

Vì `input_data` và `safe_var` được cấp phát liên tiếp trên heap với kích thước nhỏ, chúng có khả năng nằm liền kề trong bộ nhớ:

Địa chỉ	Nội dung	Thuộc về
heap+0x10	p i c o \0	input_data (5 byte)
heap+0x20	metadata chunk kế tiếp	glibc internal
heap+0x30	b i c o \0	safe_var (5 byte)

Khoảng cách chính xác giữa hai con trỏ bằng đúng kích thước chunk đầu tiên (bao gồm metadata và căn chỉnh bộ nhớ). Với request 5 byte trên hệ thống 64-bit: chunk size = $(5 + 16 + 15) \& \sim 15 = 32 = 0x20$.

5. Quy trình khai thác

1. Chạy chương trình và chọn tùy chọn ghi vào buffer.
2. Nhập chuỗi dài hơn kích thước chunk thứ nhất để tràn sang vùng nhớ **safe_var**.
3. Phần dữ liệu ghi vào vị trí của **safe_var** có thể là bất kỳ chuỗi nào khác "bico", ví dụ: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA".
4. Chọn **check_win()** để kiểm tra – khi này **strcmp(safe_var, "bico") != 0** trả về **true** và flag được in ra.

Khoảng cách tính toán thực tế

Để xác định chính xác offset từ **input_data** đến **safe_var**, cách đáng tin cậy nhất là quan sát địa chỉ mà chương trình in ra (nếu có), hoặc dùng **gdb** để kiểm tra layout heap sau khi cấp phát. Trong bài Heap0 này, khoảng cách thực tế thường là 0x20 (32 byte) hoặc 0x30 (48 byte) tùy vào libc.